

CSC343HY Practical 10

Transactions & Concurrency Control

Every query you have written so far has quietly assumed it had the database to itself. Real systems run hundreds of statements at once, and the database is allowed to *interleave* them. This practical is about the two guarantees that make interleaving safe. First, **isolation**: even though transactions overlap in time, the outcome should look as if they ran one after another, which is what **serializability** formalises. Second, **recoverability**: when a transaction aborts, the database must be able to undo it without leaving anyone who read its work stranded.

1. Transactions

The basic idea

A **transaction** is a sequence of actions that the database treats as *one unit of work*: either all of it happens, or none of it does.

Notation. Throughout this practical:

- $R(A)$: read database object A
- $W(A)$: write (to) object A
- **Commit**: the transaction finishes and its changes become permanent
- **Abort**: the transaction is rolled back as if it never ran

Subscripts identify the transaction, so $R_1(A)$ is “ T_1 reads A ”.

A single transaction is written as a list of its actions, e.g.

T1: $R(A)$ $R(B)$ $W(B)$ $W(A)$ **Commit**

Where this sits in ACID

Property	Meaning
Atomicity	All actions of a transaction happen, or none do. Delivered by Abort / rollback.
Consistency	A transaction moves the database from one valid state to another (constraints hold at the boundaries).
Isolation	Concurrent transactions do not see each other's partial work. This is the <i>serializability</i> half of the practical.
Durability	Once committed, changes survive a crash. This is why <i>recoverability</i> matters.

2. Schedules

A **schedule** S is a list of actions from a *set* of transactions, in a specific order. A schedule must preserve the internal order of each transaction: if T_1 does $R(A)$ before $W(A)$, every schedule containing both must too. What a schedule is free to choose is how the transactions are *interleaved* with one

another.

Given

T1: R(A) R(B) W(B) W(A) Commit

T2: R(B) W(A) Commit

one possible schedule is $S : R_1(A) R_1(B) R_2(B) W_2(A) W_1(B) W_1(A) \text{Commit}_1 \text{Commit}_2$, which is easier to read as a grid:

S	$time \longrightarrow$					
T1	R(A)	R(B)		W(B)	W(A)	Commit
T2			R(B)	W(A)		Commit

A **serial** schedule has no interleaving at all: every action of one transaction finishes before the next transaction starts. Serial schedules are correct by definition, but running everything serially wastes the machine. The whole game is to allow interleaving *that behaves as if it were serial*.

3. Conflicting Operations

Two operations in a schedule **conflict** if they satisfy *all three* of the following:

1. they belong to *different* transactions;
2. they access the *same* object A; and
3. at least one of them is a W(A).

Take $S_a : R_1(A) R_2(A) W_1(A) W_2(A) A_1 C_2$.

- $R_1(A)$ and $W_2(A)$ conflict; so do $R_2(A)$ and $W_1(A)$.
- $R_1(A)$ and $W_1(A)$ do *not* conflict: same transaction.
- $R_1(A)$ and $R_2(A)$ do *not* conflict: two reads never conflict.

Since two reads never conflict and same-transaction pairs never conflict, only three shapes remain, and each has its own failure mode.

Write-Read (WR): the dirty read

S	$time \longrightarrow$					
T1	R(A)	W(A)				R(B) ... Abort
T2			R(A)	W(A)	Commit	
Action	$X = A$	$A = X + 200$	$Y = A$	$A = Y * 1.05$		

T_2 reads a value of A that T_1 wrote but never committed, and T_1 then aborts. T_2 has already committed a number computed from data that, officially, never existed.

Read-Write (RW): the unrepeatable read

S	$time \longrightarrow$			
T1	R(A)		W(A)	Commit
T2		R(A)	W(A)	Commit
Action	X = 5	Y = 5	A = Y - 1	A = X - 1

T_1 read A, T_2 then changed it, and T_1 writes back a value computed from the stale copy. T_2 's update is silently thrown away.

Write-Write (WW): the blind overwrite

S	$time \longrightarrow$			
T1	W(A)		W(B)	Commit
T2		W(A)	W(B)	Commit
Action	A = 1000	A = 2000	B = 2000	B = 1000

T_1 wants A = B = 1000 and T_2 wants A = B = 2000. Either outcome alone is consistent; the schedule above ends with A = 2000 and B = 1000, which no serial execution could ever produce.

Mnemonic. RW, WR, WW are the only three error shapes. If you can classify a pair of operations into one of them, you have found the reason the schedule might be unsafe, and every edge you will draw in a precedence graph comes from exactly one of these three.

4. Serializability**Serializable**

A schedule is **serializable** if the result of executing it is identical to executing the same transactions in *some* serial order.

S	$time \longrightarrow$				
T1	R(A)	W(A)	R(B)	W(B)	
T2		R(A)	W(A)	R(B)	W(B)

This S is serializable: it is equivalent to $T_1; T_2$. T_1 's work on B is untouched by T_2 , because T_2 's operations on A cannot affect B.

Conflict serializable

“Same result” is hard to check directly, so we use a stricter but mechanical test. Two schedules are **conflict equivalent** if they involve the same actions of the same transactions *and* order every pair of conflicting operations the same way. A schedule is **conflict serializable** if it is conflict equivalent to some serial schedule.

The precedence (serializability) graph, in one line. One node per transaction. Draw an edge $T_i \xrightarrow{A} T_j$ whenever an operation of T_i on A comes before and conflicts with an operation of T_j on A. The schedule is conflict serializable iff the graph is acyclic, and any topological sort of an acyclic graph is an equivalent serial order.

Consider

S	$time \rightarrow$		
T1	R(A)	W(A)	Commit
T2		W(A)	Commit
T3		W(A)	Commit

Walking the conflicting pairs in order:

Pair	Type	Edge
$R_1(A)$ before $W_2(A)$	RW	$T_1 \xrightarrow{A} T_2$
$W_2(A)$ before $W_1(A)$	WW	$T_2 \xrightarrow{A} T_1$
$W_1(A)$ before $W_3(A)$	WW	$T_1 \xrightarrow{A} T_3$
$W_2(A)$ before $W_3(A)$	WW	$T_2 \xrightarrow{A} T_3$

The first two edges form the cycle $T_1 \rightarrow T_2 \rightarrow T_1$, so S is **not** conflict serializable.

View serializable

Conflict serializability is sufficient but not necessary: some schedules produce a serial result even though their conflict order differs. **View serializability** captures exactly the “same result” idea. Schedules S and S' are **view equivalent** when:

- (a) if T_i reads the *initial* value of object O in S , it also reads the initial value of O in S' ;
- (b) if T_i reads a value of O written by T_j in S , it also reads the value of O written by T_j in S' ;
- (c) for each object, whichever transaction performs the *final* write in S also performs the final write in S' .

S is **view serializable** if it is view equivalent to some serial schedule S' .

Return to the three-transaction schedule above and propose $S' = T_1; T_2; T_3$.

- (a) T_1 reads the initial value of A in S ; in S' it also reads the initial value.
- (b) No transaction reads a value of A written by another transaction, in either schedule.
- (c) T_3 performs the final write on A in S ; in S' it also writes last.

So the schedule *is* view serializable even though it is not conflict serializable. The two writes $W_2(A)$ and $W_1(A)$ are **blind writes** (writes not preceded by a read of the same object in that transaction), and their values are overwritten by T_3 before anyone can observe them.

Conflict serializable $\Rightarrow$ view serializable. The converse does not hold. Every view-serializable schedule that is not conflict serializable contains a blind write, which is a fast thing to look for. In practice databases test for conflict serializability, because testing view serializability is NP-complete.

5. Recoverability, ACA, and Strictness

Serializability says nothing about aborts. These three properties do. All of them are phrased in terms of *reads-from*: T_j **reads A from** T_i if T_j reads the value of A that T_i most recently wrote, and

T_i had not yet committed or aborted at that moment.

Recoverable

In a **recoverable** schedule, a transaction commits only *after* every transaction it read from has committed.

T1	R(A)	W(A)		Commit	
T2			R(A)	W(A)	Commit

T1	R(A)	W(A)		Abort	
T2			R(A)	W(A)	Abort

T_2 read **A** from T_1 , so it may not commit until T_1 does; and if T_1 aborts, T_2 must abort as well. That forced second abort is a **cascading abort**.

Unrecoverable

T1	R(A)	W(A)		Commit	
T2			R(A)	W(A)	Commit

T₂ read A from T₁ but committed before T₁ did.

T1	R(A)	W(A)		Abort	
T2			R(A)	W(A)	Commit

T₂ read A from T₁, and T₁ never committed at all.

In both cases the database is stuck: undoing T_1 would require undoing a committed transaction, which is not allowed.

Avoiding Cascading Aborts (ACA)

Recoverability still permits cascading aborts; ACA removes them. A schedule **avoids cascading aborts** if a transaction may only read data written by *committed* transactions. Equivalently: no dirty reads, ever.

T1	R(A)	W(A)	Commit		
T2				R(A)	W(A)

*ACA: T_2 waits for T_1 's commit before reading **A**.*

Strict

A schedule is **strict** if a value written by T_i is neither read *nor overwritten* by another transaction until T_i commits or aborts. Strictness is what makes undo trivial: to roll a transaction back, just restore the value it found (the “before image”), because nobody else can have touched it in the meantime.

Implications, and only in this direction:

Serial $\Rightarrow$ Strict $\Rightarrow$ ACA $\Rightarrow$ Recoverable

Strict rules out both dirty reads and dirty writes; ACA rules out only dirty reads; recoverable allows both but constrains the commit order. So $W_1(A) W_2(A) C_1 C_2$ is ACA (nothing was read) but not strict.

6. The Hierarchy of Schedules

Two independent axes are at work: *serializability* (is the outcome equivalent to a serial run?) and *recoverability* (can we still abort safely?). A schedule can be strong on one and terrible on the other.

Nesting, widest first	
Axis	
Serializability	All schedules $\supset$ View serializable $\supset$ Conflict serializable $\supset$ Serial
Recoverability	All schedules $\supset$ Recoverable $\supset$ ACA $\supset$ Strict $\supset$ Serial

Drawn as one picture, these are two chains of nested boxes that *overlap*: the “Conflict Serializable” box crosses the “Recoverable / ACA / Strict” boxes rather than sitting inside them. **Serial** is the one region that lies inside every box on both chains, which is exactly why serial execution is the gold standard for correctness (and why it is too slow to actually use).

Sanity checks on the picture. A conflict-serializable schedule need not be recoverable, and a strict schedule need not be serializable. If a claimed containment lets you deduce one axis from the other, you have misread the diagram.

7. Worked Example: Classifying Two Schedules

For each schedule, answer the same six questions: serial, serializable, conflict serializable, view serializable, recoverable, ACA.

$S_1 : R_1(A) \ R_2(A) \ W_1(A) \ W_2(A)$

S_1	time $\longrightarrow$	
T1	R(A)	W(A)
T2	R(A)	W(A)

Precedence graph. $R_1(A)$ precedes and conflicts with $W_2(A)$, giving $T_1 \xrightarrow{A} T_2$. $R_2(A)$ precedes and conflicts with $W_1(A)$, giving $T_2 \xrightarrow{A} T_1$. That is a **cycle**.

Serial? No, the actions interleave.

Conflict serializable? No, the graph has a cycle.

View serializable? No. Only two serial orders exist. In $T_1; T_2$, T_2 would read the value of **A** written by T_1 , but in S_1 it reads the initial value, violating (a)/(b). In $T_2; T_1$, the final write on **A** would be T_1 's, but in S_1 it is T_2 's, violating (c).

Serializable? No, it is not equivalent to either serial order. This is the classic lost update: whichever write lands second, one transaction's computation vanishes.

Recoverable / ACA? No dirty read occurs (both transactions read **A** before either write), so no reads-from relation constrains the commits. The schedule is not **strict**, however: $W_2(A)$ overwrites T_1 's uncommitted write.

$S_2 : R_1(A) \ R_2(B) \ W_3(A) \ R_2(A) \ R_1(B)$

S_2	$time \longrightarrow$	
T1	R(A)	R(B)
T2	R(B)	R(A)
T3	W(A)	

Precedence graph. The only write is $W_3(A)$, so every edge involves T_3 and object A: $R_1(A)$ precedes it, giving $T_1 \xrightarrow{A} T_3$; it precedes $R_2(A)$, giving $T_3 \xrightarrow{A} T_2$. The reads of B conflict with nothing. The graph $T_1 \rightarrow T_3 \rightarrow T_2$ is acyclic.

Serial? No. **Conflict serializable?** Yes. **Serializable?** Yes, equivalent to $T_1; T_3; T_2$. **View serializable?** Yes, since conflict serializable implies view serializable.

Recoverable? Cannot be determined. T_2 reads A from T_3 , so recoverability depends on whether $Commit_3$ precedes $Commit_2$, and no commits are shown.

ACA? No. T_2 reads A written by T_3 , which has not committed at that point in the schedule. That is a dirty read.

Convention for missing commits. If a schedule shows no *Commit* or *Abort*, you cannot assume anything about the commit order; the safe reading is that each transaction commits somewhere at the end of its own row. Properties that depend on that order (recoverable) become “not determined”, while properties that depend only on the reads (ACA, strict) can still be decided. Always state which reading you used.

8. Exercises

Show your reasoning, not just the final answer.

- For the schedule $S : R_1(A) \ W_2(A) \ R_2(B) \ W_1(B)$, list every pair of conflicting operations, classify each as RW, WR, or WW, and say why $R_1(A)$ and $R_2(B)$ are not a conflicting pair.
- Draw the precedence graph for each schedule and decide whether it is conflict serializable. If it is, give an equivalent serial order.
 - $S_a : R_1(A) \ W_2(A) \ R_3(A) \ W_1(B) \ R_2(B) \ W_3(B)$
 - $S_b : R_1(A) \ W_2(A) \ R_2(B) \ W_1(B)$
- Consider $S : R_1(X) \ W_2(X) \ W_1(X) \ W_3(X)$, with commits in the order C_1, C_2, C_3 at the end.
 - Show that S is not conflict serializable.
 - Show that S is view serializable by proposing a serial schedule and checking all three view-equivalence conditions.
 - What feature of S makes this gap between the two definitions possible?
- For each schedule below, state whether it is recoverable, whether it avoids cascading aborts, and

whether it is strict. Justify each answer in one sentence.

(a) $S_a : R_1(A) W_1(A) R_2(A) W_2(A) C_1 C_2$

(b) $S_b : R_1(A) W_1(A) R_2(A) W_2(A) C_2 C_1$

(c) $S_c : W_1(A) W_2(A) C_1 C_2$

(d) $S_d : R_1(A) W_1(A) C_1 R_2(A) W_2(A) C_2$

5. An account starts at $A = 100$. T_1 deposits \$50; T_2 deposits \$20. They interleave as:

T1	R(A)		W(A)	
T2	R(A)		W(A)	
Action	X = A	Y = A	A = X + 50	A = Y + 20

(a) What is the final value of A ? What should it have been?

(b) Which conflict type is responsible, and is the schedule conflict serializable?

(c) Describe, in one sentence, a locking rule that would have prevented it.

6. True or false, with a one-line justification or counterexample for each.

(a) Every conflict-serializable schedule is view serializable.

(b) Every view-serializable schedule is conflict serializable.

(c) Every serial schedule is strict.

(d) Every recoverable schedule avoids cascading aborts.

(e) Every conflict-serializable schedule is recoverable.

9. Common Pitfalls

- Drawing an edge for every pair of operations.** Only pairs that satisfy all three conflict conditions produce an edge; two reads, or two operations of the same transaction, never do.
- Getting the edge direction backwards.** The edge always points from the operation that happens *earlier* to the one that happens later.
- Concluding “not serializable” from a cycle.** A cycle proves only that the schedule is not *conflict* serializable; it may still be view serializable, as blind-write examples show.
- Confusing the two axes.** Serializability is about the outcome; recoverability is about aborts. Neither implies the other.
- Forgetting that ACA does not mention writes.** ACA forbids dirty reads only. Ruling out dirty *writes* as well is what makes a schedule strict.
- Inventing commits that are not in the schedule.** If no commit or abort appears, do not assume an order; say which properties are therefore undetermined.

Before You Leave

Checklist

1. I can write a schedule in both list and grid form, and say what makes it serial.
2. I can apply the three conflict conditions and classify any conflicting pair as RW, WR, or WW.
3. I can build a precedence graph and use it to decide conflict serializability and to read off a serial order.
4. I can check the three view-equivalence conditions, and explain why blind writes create the gap between conflict and view serializability.
5. I can decide whether a schedule is recoverable, ACA, and strict, and explain what goes wrong when each fails.
6. I can place any schedule in the hierarchy diagram and justify why the serializability and recoverability axes are independent.